

Personalized Linux, and Git Cheat Sheet

ITEC 111 - Online Activity #2

1. Basic Linux Commands

Command	Sample usage	Description
pwd	pwd	Tells you where you currently are in the terminal (your current folder).
ls	ls -la	Shows what's inside a folder — with -la you also see hidden files and extra details.
cd	cd Documents	Lets you move into another folder.
mkdir	mkdir new_folder	Creates a brand new, empty folder.
rm	rm file.txt	Deletes a file for good. Add -r if you're deleting a whole folder.
cp	cp a.txt backup.txt	Makes a copy of a file (or folder, with -r) somewhere else.
mv	mv a.txt docs/	Moves a file to another folder, or just renames it.
touch	touch notes.txt	Quickly creates an empty file, like a placeholder.
cat	cat notes.txt	Dumps the whole content of a file straight onto your screen.
chmod	chmod +x run.sh	Changes who can access/run a file, e.g. making it executable.
grep	grep "error" log.txt	Searches inside a file for lines that match a certain word or pattern.
sudo	sudo apt update	Runs a command as admin/root, for when you need higher access.

2. Basic Git Commands

Command	Sample usage	Description (own words)	Common flags
git init	git init	This is the first thing you do — it turns your current folder into a Git repo, basically "activating" it.	—
git clone	git clone <url>	Downloads/copies an existing repo (with its whole history) onto your computer.	--depth 1
git status	git status	Acts like a checker — tells you which files were changed, staged, or not tracked yet.	-s
git add	git add index.html	Prepares a changed file before committing — like putting it in a "cart" before checkout.	-A, .
git commit	git commit -m "fix bug"	Permanently saves your staged changes into the history, with a message describing what you did.	-m, -am
git push	git push origin main	Sends your commits up to GitHub (or wherever the remote repo is hosted).	-u, --force
git pull	git pull origin main	Opposite of push — grabs and merges the latest changes from the remote into your local copy.	--rebase
git branch	git branch feature1	Used to view, create, or delete branches (basically separate "versions" of the project).	-d, -a
git checkout	git checkout feature1	Switches you to another branch, or can restore a file to an older version.	-b
git merge	git merge feature1	Combines the changes from one branch into the branch you're currently on.	--no-ff
git log	git log --oneline	Shows the history of all your commits, like a project logbook.	--graph
git remote	git remote -v	Shows (or lets you manage) where your repo is connected to, e.g. a GitHub link.	add

3. Git's Three-Tree Architecture

Git basically manages your project through three "stages" that your files pass through before they're permanently saved:

1. Working Directory — this is your actual folder, where you can see and edit files in real time. Still just a "draft" at this point.

2. Staging Area (Index) — this is where you "park" the specific changes you want to include in your next commit. Files land here once you run `git add`.

3. Repository (.git folder) — this is where the project's history is permanently stored. Running `git commit` takes whatever is in the staging area and turns it into a permanent snapshot here.

Flow: Working Directory --(`git add`)--> Staging Area --(`git commit`)--> Repository

References

GeeksforGeeks. (n.d.). Basic Linux Commands. <https://www.geeksforgeeks.org/linux-unix/basic-linux-commands/>

GeeksforGeeks. (n.d.). Basic Git Commands With Examples. <https://www.geeksforgeeks.org/git/basic-git-commands-with-examples/>

Sherke, R. (n.d.). Git Three-Stage Architecture. Medium. <https://medium.com/@rituserke86/git-three-stage-architecture-2400d5cbfdfe>